



DETECÇÃO DE TÓPICOS EM CHAMADOS DE SUPORTE TÉCNICO

Integrantes grupo 8:

- Gabriel Fernando De Jesus Silva
- Giovana Ferreira Santos
- Leonardo Lopes Cruz
- Pedro Henrique dos Santos Ferreira
- Vitor Carvalho Pereira

DESCRIÇÃO DO PROBLEMA

O Problema

Equipes de TI recebem grandes volumes de chamados de suporte técnico.

Categorizar manualmente é lento e sujeito a erros.

Como descobrir automaticamente quais problemas são mais recorrentes?

Dataset: 60 chamados fictícios em 6 categorias:
Rede/VPN, Banco de Dados, Autenticação, E-mail,
Hardware, Software

Objetivo

- 1 Agrupar automaticamente chamados por assunto
- 2 Descobrir tópicos recorrentes sem lista prévia
- 3 Usar grafos de coocorrência + algoritmos clássicos
- 4 Avaliar qualidade com pureza e modularidade

MODELAGEM DO GRAFO



Vértice

Uma palavra-chave (substantivo, nome próprio ou adjetivo) extraída dos chamados via spaCy



Aresta

Liga duas palavras que aparecem no mesmo chamado (grafo não-direcionado)



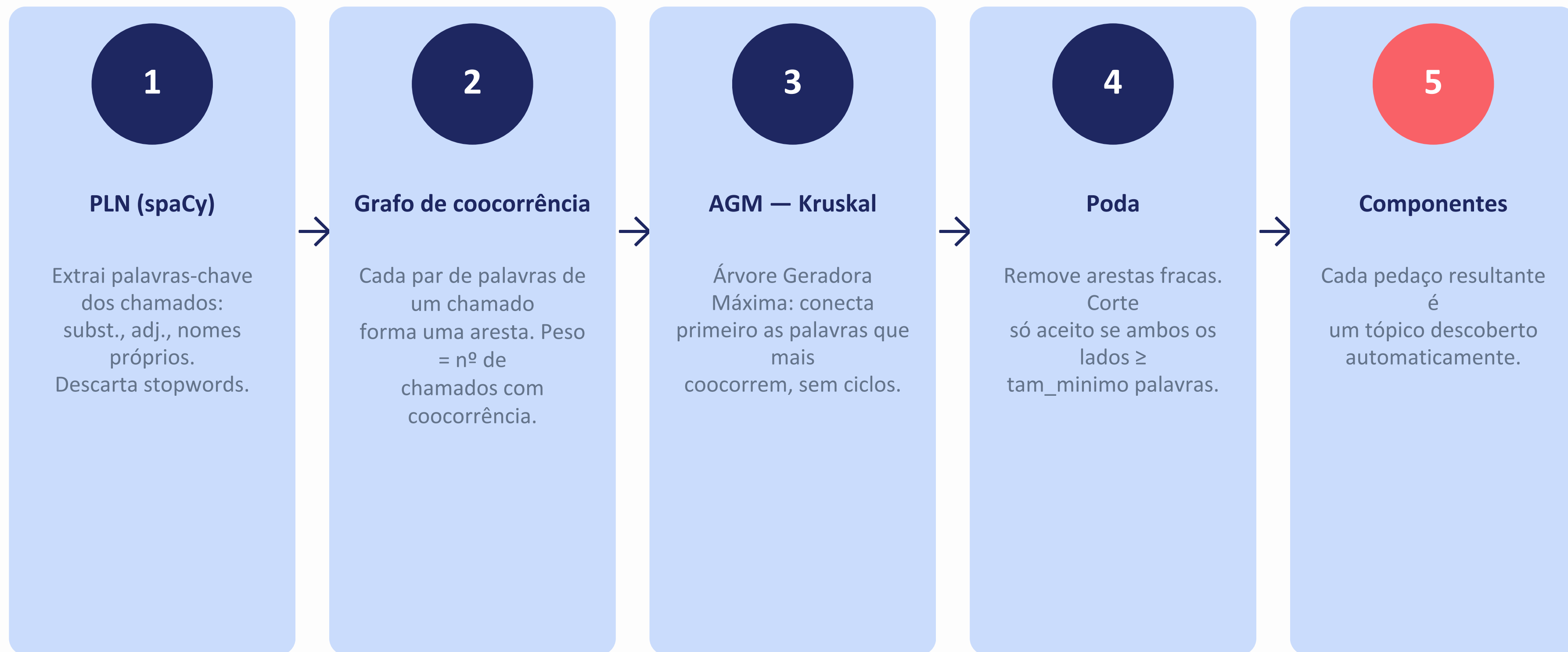
Peso

Número de chamados em que as duas palavras coocorrem — quanto maior, mais relacionadas

Grafo não-direcionado · ponderado · representado por lista de adjacência (src/grafos.py)

Exemplo: "banco" e "timeout" coocorrem em 4 chamados → aresta com peso 4

DESCRIÇÃO DA SOLUÇÃO



Union-Find com compressão de caminho: estrutura adicional usada no Kruskal e na formação dos componentes

IMPLEMENTAÇÃO

Módulos do projeto

`grafo.py` Classe Grafo (lista de adjacência) + Union-Find

`comunidades.py` Kruskal (AGM), poda e componentes

`pln.py` Extração de palavras-chave com spaCy

`analise.py` Pureza e modularidade (métricas)

`visualizacao.py` construção do dashboard interativo

Algoritmos de grafo

Kruskal, Union-Find, BFS de componentes — implementados pelo grupo

spaCy para PLN

Tokenização e filtragem de palavras-chave para construção do grafo

Poda parametrizável

Controla granularidade dos tópicos; ajustável em `main.py` e `analisar.py`

Dashboard interativo

HTML/JS com `vis.js` — exibe grafo, AGM, arestas podadas e tópicos

Dependências: Python 3 · spaCy (pt_core_news_sm) · Nenhuma lib de grafos externa

EXEMPLOS DE ENTRADA E SAÍDA

ENTRADA — chamado JSON

```
{ "id": 11, "texto": "O banco de dados PostgreSQL está muito lento e retornando timeout nas consultas." }
```

PLN extrai palavras-chave:

"banco", "dados", "postgresql", "lento", "timeout", "consultas"

Chamado classificado em:

Tópico #3 — Banco de Dados

Ex. Tópicos descobertos (tam_minimo = 6)

Tópico #3 · Banco de Dados

banco, servidor, dados, aplicação,
postgresql...

Tópico #4 · Rede / VPN

conexão, vpn, instável, internet, wi-fi,
roteador...

Tópico #6 · E-mail

outlook, e-mails, assinatura, filtro, spam,
anexos...

ANÁLISE DOS RESULTADOS

167Vértices
(palavras-chave)**518**Arestas
no grafo**15**Tópicos
descobertos**80%**Pureza
vs. gabarito**0,57**Modularidade
(Q)

Efeito do parâmetro tam_minimo

tam_minimo	Nº tópicos	Pureza	Modularidade Q
4	23	86,7%	0,505
6	15	80,0%	0,569
8	13	71,7%	0,582 ★
10	11	70,0%	0,580
15	9	55,0%	0,567

Modularidade máxima em tam_minimo = 8 · Menor tam_minimo → mais tópicos e maior pureza, mas menos coesão

COMO O LLM FOI UTILIZADO NO DESENVOLVIMENTO DO TRABALHO?

**DB**

Geração da base de dados

Os 60 chamados fictícios de suporte técnico em `data/chamados.json` foram gerados por LLM, cobrindo 6 categorias com sobreposição intencional de vocabulário para tornar a detecção não-trivial.

**{ }**

Apoio ao desenvolvimento

Organização do código em módulos, estrutura de pastas e documentação do projeto (README e comentários inline) contaram com auxílio de LLM.

A modelagem do grafo e todos os algoritmos principais (Kruskal, Union-Find, poda, componentes) foram implementados sem uso de bibliotecas prontas de grafos.

CONCLUSÃO



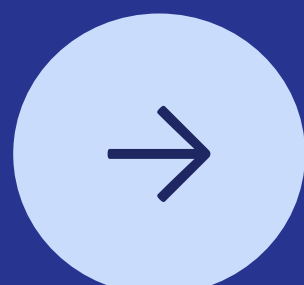
Os 60 chamados fictícios de suporte técnico em data/chamados.json foram gerados por LLM, cobrindo 6 categorias com sobreposição intencional de vocabulário para tornar a detecção não-trivial.



Tensão pureza × modularidade: cortar mais gera tópicos puros porém menos coesos; máxima modularidade em tam_minimo = 8.



Limitações: Hardware se espalha em 5 tópicos (vocabulário variado); chamados curtos compartilham palavras genéricas.



Próximos passos: uso de embeddings semânticos e algoritmos de comunidade adicionais (Louvain, Girvan-Newman).